

NcedoCare

Smarter care for stronger communities

React.js Web App Frontend Framework Recommendation

Prepared for: Sibahle Dube

University of Johannesburg - Sustainathon 2026

Date: July 2026

1. Overview & Context

NcedoCare's original system design specified React Native for a cross-platform patient mobile app. The staff-facing portal (Admin, Doctor, Nurse dashboards) is instead being built as a web application using React.js, based on the delivered wireframes covering authentication, triage review, escalation, audit logging, and facility management screens.

This document consolidates the recommended frontend framework and library stack for the React JS web app, aligned with the existing Firebase backend, Gemini/OpenAI AI layer, WebSocket voice communication, and blockchain accountability layer defined in the NcedoCare System Design.

Why Next.js Over a Plain Vite + React SPA

Given the staff portal handles role-based dashboards (Admin, Doctor, Nurse), session timeouts, and Access-Denied gating for sensitive health data, a framework with built-in routing and server-side route protection is preferred over a client-only single-page app.

Aspect	Next.js	Vite + React (SPA)
Routing	File-based, built-in, ideal for multi-role dashboards	Requires manual setup via React Router
Auth / SSR	Server-side rendering enables protected routes; more secure for health data	Client-only; auth checks happen post-load (flash of unauthenticated content)
API Routes	Built-in API routes can securely proxy Firebase Admin SDK calls	Requires a separate backend or Cloud Functions call
Performance	Automatic code splitting, image optimization out of the box	Faster dev server, but code splitting is manual
Firebase Fit	Works well with Firebase Hosting + Cloud Functions, or Vercel	Works well with Firebase Hosting